Participant Guide: AI Hackathon

State, Local Government and Higher Education

This guide gets you from a bare laptop to a working AWS deployment, driven by an AI agent. It is written for participants who are new to AWS, new to Kiro, or both.

You can read this guide offline. Everything you need for the three days is here, in order.

About this guide

Who this is for

You are attending a three-day hackathon. Your team gets a temporary AWS account and a Kiro Pro license. You are going to build a working prototype of something that solves a real problem in your agency or institution.

You do not need prior AWS or Kiro experience. You do need to be comfortable in a terminal and comfortable reading code that somebody else wrote — because for most of this event, Kiro is going to write it.

The 72-hour clock

Important

Your workshop AWS account is available for **72 hours**. When that window closes, the workshop environment shuts down and the account and everything in it is deleted. There is no extension and no recovery.

Nothing you build survives automatically. Anything you want to keep must be pushed to **your own GitHub repository** or saved to **your local machine** before the window closes. Set up that destination on day one, not on day three — see Part 8: Save your work and clean up.

How to use this guide

Work through Parts 1 through 5 in order before the event, or in the first couple of hours of day one. They are sequential and each one depends on the last. Parts 6 and 7 are reference material you will come back to across all three days. Read Part 8 on day one so the ending does not surprise you.

If you only have twenty minutes, do Part 1, Part 2, and Part 4. That gets you a working agent with AWS access, which is the minimum viable setup.

Two sets of credentials, and why that matters

This is the single most common point of confusion, so it comes first.

Your event organizer gives you **two separate sets of credentials** that do different things and live in different places:

Credentials	What they unlock	Where you use them	Region
Kiro Pro (`KiroProStartURL`, `KiroProUsername`,	Your Kiro license — the AI agent itself	Kiro IDE and Kiro CLI sign-in only	`us-east-1`

`KiroProPassword`, `KiroProRegion`)			
AWS account access (from the Workshop Studio dashboard)	Your team's temporary AWS account	AWS Management Console, AWS CLI, and Kiro's AWS tools	The event Region, typically `us-west-2`

They are not interchangeable. Kiro Pro credentials will not get you into the AWS Console, and AWS account credentials will not sign you in to Kiro. Signing in to Kiro does **not** give Kiro access to your AWS account — that is a separate step, covered in Part 5.

Important

The IAM Identity Center instance provisioned for this event is an **account instance whose only job is issuing your Kiro Pro license.** It contains no AWS account permission sets. That means the Kiro Pro start URL is useless for AWS access, and `aws configure sso` has nothing to enumerate. Get AWS credentials from the Workshop Studio dashboard instead, as described in Part 4.

Note

The Kiro Pro identity directory is hosted in `us-east-1` regardless of which Region your event runs in. When Kiro asks for a Region during sign-in, enter the value from `KiroProRegion` (`us-east-1`), not your event Region.

Document conventions

Command examples use a `$` prompt for macOS and Linux, and a `PS>` prompt for Windows PowerShell. Do not type the prompt.

Values in double braces are placeholders you replace with your own values, for example `{{your-profile-name}}`.

Part 1: Install Kiro on your workstation

Install both the Kiro IDE and the Kiro CLI. They are two front ends onto the same agent and they share the same configuration, so anything you set up in one applies to the other. You will want both: the IDE for building, the CLI for anything involving command output.

Topics

Prerequisites (Part 1)

Confirm your workstation meets the system requirements.

Kiro IDE

- macOS (Intel or Apple Silicon)
- Windows 10 or 11 (64-bit)
- Linux: Ubuntu 24+, Debian 13+, Fedora 40+, Arch, or Mint 22+

Kiro CLI

- macOS
- Windows 11 (PowerShell)
- Linux with glibc 2.34 or later, or a musl variant

Important
If your organization restricts software installation on managed devices, do not fight it. Your event provides browser-based alternatives that need nothing installed locally — a hosted VS Code environment and a hosted Kiro desktop. Ask a facilitator for the `VScodeIdeURL` or `KiroRDPConnectionURL` output, then skip to Part 2. This is a common situation in public sector IT and there is no penalty for it.

Step 1: Install the Kiro IDE

To install the Kiro IDE

1. Open kiro.dev/downloads in your browser.
2. Download the installer for your operating system.
3. Run the installer:
   - **macOS** — open the `.dmg` and drag Kiro to your Applications folder.
   - **Windows** — run the `.exe` and follow the prompts.
   - **Linux** — install the `.deb` or `.rpm` package with your package manager.
1. Launch Kiro. Leave the sign-in screen open; you will complete it in Part 2.
2. When prompted, you can import your existing Visual Studio Code settings and extensions. If you use a different editor, skip this and pick a theme.

Step 2: Install the Kiro CLI

To install the Kiro CLI on macOS or Linux

1. Run the install script:

```
$ curl -fsSL https://cli.kiro.dev/install | bash
```

1. On Linux, the CLI installs to `$HOME/.local/bin`. If the installer reports that this directory is not on your `PATH`, add it to your shell configuration file:

```
$ export PATH="$HOME/.local/bin:$PATH"
```

Add that same line to `~/.zshrc` or `~/.bashrc` to make it permanent.

1. Verify the installation:

```
$ kiro-cli --version
```

To install the Kiro CLI on Windows

1. Open PowerShell and run:

```
PS> irm 'https://cli.kiro.dev/install.ps1' | iex
```

1. Close and reopen PowerShell so the updated `PATH` takes effect.
2. Verify the installation:

```
PS> kiro-cli --version
```

Note
If you previously installed the Amazon Q Developer CLI, the installer detects it and updates it to Kiro CLI in place.

Step 3: Install the supporting tools

You need three more things for the AWS work in Parts 4 through 6. Install them now so you are not stopping to do it later.

Tool	Why you need it	Install
AWS CLI v2	Authenticating to AWS, and everything Kiro does with your account	Installing or updating to the latest version of the AWS CLI
uv	Runs the AWS MCP server that connects Kiro to AWS	Installing uv
Node.js (LTS)	Required for the AWS CDK, and for most web frameworks	nodejs.org

To verify all three

```
$ aws --version
$ uv --version
$ node --version
```

Important
Install AWS CLI version **2.35.0 or later**. Part 5 uses `aws configure agent-toolkit`, which was added in 2.35.0 and is by far the easiest way to connect Kiro to AWS. If `aws --version` reports anything lower, update before continuing.

Part 2: Sign in to Kiro with your workshop credentials - (Prior to event use "Google, github or builder ID)

Your event provides a Kiro Pro license through AWS IAM Identity Center. Sign in with the organization identity option, not with a personal Google, GitHub, or AWS Builder ID account.

Topics

Prerequisites (Part 2)

- Kiro IDE and Kiro CLI installed, from Part 1.
- Access to your event's Workshop Studio dashboard.

Step 1: Collect your Kiro Pro values

To find your Kiro Pro credentials

1. Sign in to Workshop Studio with the event access code your organizer gave you.
2. Open the event dashboard and locate these four outputs. Copy them somewhere you can paste from.

Output	Example value	What it is
`KiroProStartURL`	`https://d-xxxxxxxxxx.awsapps.com/start`	Your IAM Identity Center start URL
`KiroProRegion`	`us-east-1`	The Region hosting the identity directory
`KiroProUsername`	`kiro`	Your user name
`KiroProPassword`	*(generated string)*	A **one-time** password

Important

`KiroProPassword` is a one-time password. The first time you sign in, you are prompted to set a new password. Choose something you will remember, or write it down — you need it again in Step 3 for the CLI, and the original one-time value stops working once used.

Step 2: Sign in to the Kiro IDE

To sign in to the Kiro IDE

1. In the Kiro welcome screen, choose **Sign in with your organization identity** (it may appear as **Your organization**).
2. When prompted, choose the option to sign in through **IAM Identity Center**.
3. Enter the **Start URL** (`KiroProStartURL`) and the **Region** (`KiroProRegion`, which is `us-east-1`), then choose **Continue**.
4. Kiro opens your default browser. If prompted to allow Kiro to open an external site, choose **Open**.
5. Sign in with `KiroProUsername` and `KiroProPassword`. Set a new password when prompted.
6. Choose **Allow access**.

7. Return to Kiro. To confirm your license is active, select your user icon and verify that a Kiro Pro subscription is shown.

Step 3: Sign in to the Kiro CLI

To sign in to the Kiro CLI

1. Open a terminal and run:

```
$ kiro-cli login
```

1. Choose **Sign in with your organization identity**.
2. Enter the **Start URL** (`KiroProStartURL`) and the **Region** (`us-east-1`).
3. Copy the authentication URL shown in the terminal into your browser.
4. Sign in with your user name and the password you set in Step 2, then choose **Allow access**.
5. Return to the terminal and start the agent:

```
$ kiro-cli
```

To verify both surfaces work

Ask the agent a question that requires no tools:

```
Explain what you can do in this project and what configuration you can see.
```

If you get a coherent answer in both the IDE and the CLI, Part 2 is complete.

Troubleshooting sign-in

The password is rejected at the Identity Center sign-in screen

The one-time password was probably already used. Generate a fresh one:

1. Open the AWS Management Console and switch the Region to **US East (N. Virginia)** — `us-east-1`.
2. Search for and open **IAM Identity Center**.
3. In the navigation pane, choose **Users**, then choose the **kiro** user.
4. Choose **Reset password**, then **Generate a one-time password**.
5. Sign in with the new value.
6. Switch the Console Region back to your event Region before continuing.

Sign-in succeeds but no Pro subscription appears

The license assignment may not have completed. Ask a facilitator to verify the Kiro subscription on the `kiro` user in IAM Identity Center.

The browser does not open

Use the device flow, which prints a URL and code you can open on any device:

```
$ kiro-cli login --use-device-flow
```

Part 3: AWS fundamentals

If you have used AWS before, skim the Guardrails on your event account section and move on. If you have not, this section gives you the mental model you need for the rest of the day.

Topics

The five concepts that matter most

Account. A container holding your resources, your bill, and your permission boundaries. Your team has one temporary account for the event. It is deleted after the event.

Region. A physical geographic location containing AWS data centers, for example `us-west-2` (Oregon). Regions are isolated from each other. A resource you create in `us-west-2` is invisible from `us-east-1`, so **if something you created has apparently vanished, check the Region selector first**. Most "access denied" and "not found" errors at hackathons are a Region mismatch, not a permissions problem.

IAM. Identity and Access Management: who can do what to which resources. You will not need to write much IAM this week, but you will see it. Two terms worth knowing:

- A **policy** is a document listing allowed and denied actions.
- A **role** is an identity that a service assumes to act on your behalf. When your Lambda function reads from an S3 bucket, it does so using a role.

The API is the product. Everything in AWS is an API call. The Console, the CLI, the SDKs, the CDK, and Kiro's AWS tools are all just different ways to make the same calls. This is why an agent can operate AWS at all, and it is why anything you can do by clicking, you can also do by asking.

Managed and serverless. Prefer services where AWS runs the infrastructure. You have three days and a demo at the end of them. Do not spend that time patching an operating system. In practice this means Lambda over EC2, DynamoDB over a self-managed database, and Amazon Bedrock over hosting your own model.

Services you are most likely to use

You have access to the broad AWS catalog. These are the ones that come up repeatedly in public sector prototypes.

Service	What it does	Typical use in this hackathon
Amazon Bedrock	Managed access to foundation models through one API	Summarizing, extracting, classifying, answering questions, plain-language rewriting
AWS Lambda	Runs your code without servers	Nearly all application logic

Amazon S3	Object storage	Documents, uploads, static websites, data sets
Amazon DynamoDB	Serverless NoSQL database	Application state, queues of work items, results
Amazon API Gateway	Managed HTTP APIs	The front door between your web app and your Lambda functions
Amazon Textract	Extracts text, forms, and tables from documents	Any workflow that starts with a scanned PDF
Amazon Transcribe	Speech to text	Meeting minutes, call recordings, public hearings
Amazon Translate	Machine translation	Multilingual public-facing services
AWS Step Functions	Coordinates multi-step workflows	Document pipelines with several stages
Amazon CloudWatch	Logs, metrics, and alarms	Finding out why your Lambda function failed

A very common and very effective architecture for this event: **S3 for input, Lambda for processing, Bedrock for the intelligence, DynamoDB for results, API Gateway for access.** It is serverless, it costs almost nothing at demo scale, and Kiro can scaffold the whole thing.

Guardrails on your event account

Your account has broad permissions with a few deliberate blocks. You do not need to work around these; they exist to prevent expensive accidents.

Blocked actions include:

- Large, GPU, and specialized EC2 instance types (anything `6xlarge` and above, plus `g*`, `p*`, `x*`, and bare metal)
- AWS Organizations, Control Tower, and Firewall Manager
- Purchasing Reserved Instances, Savings Plans, or capacity reservations
- Registering or transferring Route 53 domains
- Service quota increase requests
- AWS Support cases and Trusted Advisor
- Third-party AWS Marketplace machine images
- Account-level modifications and S3 Object Lock or Backup Vault Lock

Note
If an action fails with an explicit `Deny`, check this list before debugging. And if you find yourself needing a GPU instance for a hackathon prototype, that is a strong signal to use a managed service such as Amazon Bedrock instead.

Data handling rules

Warning

Use synthetic and public data only. Do not introduce any real constituent, resident, student, patient, or employee records into your event account — including de-identified extracts, small samples, and single files "just to test the parser."

For this audience, the prohibition explicitly includes student education records covered by FERPA, criminal justice information, health and human services case records, tax and revenue records, voter registration files, HR and payroll data, utility account data, and anything received under a data sharing agreement.

Safe sources for realistic inputs:

- Your organization's open data portal, or data.gov
- Published public records — municipal codes, board minutes, course catalogs, policy manuals
- Public statistical data such as IPEDS, NCES, and Census
- Synthetic data you generate. Kiro will produce a realistic test corpus matching your real schema in about a minute:

```
Generate 25 synthetic permit applications as JSON that match this schema.
Vary them realistically: some complete, some missing required fields, some
with formatting inconsistencies. Label the file clearly as synthetic.
```

Part 4: Connect the AWS CLI to your workshop account

Configuring the AWS CLI is the foundational step for everything that follows. The CDK, the SDKs, and Kiro's AWS tools all read the credentials the AWS CLI stores, so you configure once and everything else inherits it.

Topics

Prerequisites (Part 4)

- AWS CLI version 2.35.0 or later installed. Verify with `aws --version`.
- Access to your event's Workshop Studio dashboard.

Important

Do not run `aws configure sso` for this event, and do not try to use your Kiro Pro start URL for AWS CLI access.

The IAM Identity Center instance provisioned for this workshop is an **account instance** that exists for one purpose: issuing your Kiro Pro license. It has no AWS account permission sets, so there is nothing for `aws configure sso` to enumerate and no Identity Center profile you can build for CLI access. Your AWS account credentials come from the Workshop Studio dashboard instead, as described below.

Step 1: Sign in to the AWS Management Console

Do this first. Both credential options in Step 2 depend on it, and the Console is also how you visually confirm anything

you build.

To open the AWS Console

1. Sign in to Workshop Studio with the event access code your organizer gave you.
2. In the left navigation of the event dashboard, choose the link to open the **AWS Console**. It signs you straight in to your team's account — there is no user name or password to enter.
3. Confirm two things in the Console header before going further:
   - The **account number** at the top right matches what your teammates see. If it does not, one of you is in the wrong account.
   - The **Region** selector shows your event Region, typically **US West (Oregon)** — `us-west-2`.

Step 2: Get credentials for the AWS CLI

You have two options. Start with Option A, which always works. Option B is worth switching to once you are set up, because it stops you re-pasting keys over a three-day event.

Option	How it works	Session length
A: Copy short-term credentials	Paste three values from the Workshop Studio panel into your terminal	Fixed; you re-copy when it expires
B: `aws login`	Reuses your browser Console session to issue credentials	Rotates automatically every 15 minutes, valid up to 12 hours

Option A: Copy short-term credentials

To use short-term credentials as environment variables

1. In the Workshop Studio event dashboard, open the AWS access panel in the left navigation and locate the AWS CLI credentials.
2. Copy the credentials block for your operating system and paste it into your terminal. It sets three environment variables:

```
$ export AWS_ACCESS_KEY_ID="{{ASIA...}}"
$ export AWS_SECRET_ACCESS_KEY="{{...}}"
$ export AWS_SESSION_TOKEN="{{...}}"
```

On Windows PowerShell:

```
PS> $env:AWS_ACCESS_KEY_ID="{{ASIA...}}"
PS> $env:AWS_SECRET_ACCESS_KEY="{{...}}"
PS> $env:AWS_SESSION_TOKEN="{{...}}"
```

1. Set your default Region so you do not have to pass it on every command:

```
$ export AWS_DEFAULT_REGION={{us-west-2}}
```

On Windows PowerShell:

```
PS> $env:AWS_DEFAULT_REGION="{{us-west-2}}"
```

Important
Environment variables apply only to the terminal session where you set them. Open a new terminal and you must set them again. This gets old quickly across three days, which is why Option B exists.

Warning
Never paste these values into a source file, a Git repository, a chat message, or an issue tracker. They grant full access to your team's account for as long as they are valid. Be especially careful if you are pushing work to a public GitHub repository.

Option B: Use `aws login`

`aws login` issues short-term credentials that rotate automatically every 15 minutes and stay valid for up to 12 hours, with nothing written to disk as a long-lived secret. **It works for this event as long as you have already signed in to the AWS Console in your browser**, per Step 1 — it reuses that session rather than asking you to authenticate again.

To get credentials with `aws login`

1. Confirm you are signed in to the AWS Console in your browser. If you are not, go back to Step 1.
2. Run:

```
$ aws login
```

1. Your browser opens and completes the request against your existing Console session. Approve it if prompted.
2. Set your Region if you have not already:

```
$ export AWS_DEFAULT_REGION={{us-west-2}}
```

To keep these credentials separate from anything else on your machine, use a named profile instead:

```
$ aws login --profile {{hackathon}}
$ export AWS_PROFILE={{hackathon}}
```

On Windows PowerShell:

```
PS> $env:AWS_PROFILE = "{{hackathon}}"
```

Note
`aws login` requires AWS CLI 2.32.0 or later. If there is no browser on the machine, `aws login --remote` prints a URL and code you can use from another device. `aws login` is not available in the AWS GovCloud (US) or China Regions, which may matter when you take this technique back to your own environment.

Tip

If `aws login` fails for any reason, fall back to Option A and keep building. Do not spend hackathon time debugging your credential method.

Verify your access

Confirm three things regardless of which option you used: who you are, which Region you are in, and that you can actually call a service.

```
$ aws sts get-caller-identity
```

The response shows your account number and the ARN of the identity you are using:

```
{
    "UserId": "AROAEXAMPLEID:participant",
    "Account": "123456789012",
    "Arn": "arn:aws:sts::123456789012:assumed-role/WSParticipantRole/participant"
}
```

Check your Region and list something harmless:

```
$ aws configure get region
$ aws s3 ls
```

An empty response from `aws s3 ls` is a success — it means you authenticated and the account has no buckets yet.

Note
Confirm the account number matches what your teammates see, and matches what you saw in the Console in Step 1.

Renewing access during the event

Your credentials expire well before the event does. Over three days you will re-authenticate several times. This is normal and it is not a sign anything is broken.

You will know it is time when commands that worked an hour ago start returning `ExpiredToken` or `The security token included in the request is invalid`.

To renew

- **Option A** — return to the Workshop Studio AWS access panel and copy a fresh credentials block into your terminal.
- **Option B** — make sure your browser Console session is still valid, then run `aws login` again.

Important
When your AWS credentials expire, Kiro's AWS tools stop working too, because they use the same credentials. After renewing, reconnect the AWS MCP server from the MCP Servers view in Kiro. See Part 5.

Troubleshooting credentials

Symptom	Cause and fix
`Unable to locate credentials`	No credentials in this shell. Re-paste your short-term

	credentials, or run `aws login`.
`ExpiredToken` or `The security token included in the request is invalid`	Session expired. See Renewing access during the event.
`aws configure sso` finds no accounts or roles	Expected. The event Identity Center instance is for Kiro licensing only and has no AWS permission sets. Use Step 2 instead.
`aws login` opens the browser and fails	Your Console session has probably lapsed. Sign in to the Console again per Step 1, then retry. If it still fails, use Option A.
`AccessDenied` on something that should work	Check your Region first with `aws configure get region`. If the Region is right, check the guardrails list.
Resources you created are missing	Region mismatch. You are looking in a different Region from the one you deployed to.
Commands work in one terminal but not another	Environment variables are per-session. Set them again in the new terminal, or use `aws login` with a named profile and `AWS_PROFILE`.
`aws: command not found`	AWS CLI not installed or not on `PATH`. See Installing or updating to the latest version of the AWS CLI.

Part 5: Connect Kiro to your AWS account

Signing in to Kiro gave you the agent. This part gives the agent hands: the ability to read live AWS documentation, inspect resources in your account, deploy, and troubleshoot.

The connection is made through the **AWS MCP Server**, a managed remote server hosted by AWS. It authenticates with the AWS credentials you configured in Part 4 using IAM SigV4, which means no credentials are exposed to the agent, every call is validated before execution, and everything is logged in AWS CloudTrail.

Topics

Prerequisites (Part 5)

- Working AWS credentials from Part 4. Confirm with `aws sts get-caller-identity`.
- `uv` installed, from Part 1, Step 3.
- AWS CLI 2.35.0 or later for Method A.

Method A: Use the setup wizard

This is the recommended path. One command detects your installed coding agents, configures the AWS MCP Server connection, and installs a curated set of AWS agent skills.

To run the setup wizard

1. Run:

```
$ aws configure agent-toolkit
```

1. Follow the prompts. The wizard detects Kiro and writes the MCP configuration for you.
2. Restart Kiro, or reconnect the MCP server from the MCP Servers view in the Kiro feature panel.

The skills installed alongside the server give Kiro validated, current guidance for specific AWS tasks — CDK authoring, CloudFormation troubleshooting, serverless patterns, IAM, cost analysis — which it loads only when relevant.

Method B: Configure MCP manually

Use this if the wizard is unavailable or your AWS CLI is older than 2.35.0.

To configure the AWS MCP Server manually

1. Open your Kiro MCP configuration file. Use `~/.kiro/settings/mcp.json` for all projects, or `.kiro/settings/mcp.json` inside a project for that project only.
2. Add the `aws-mcp` server. Set the `AWS_REGION` metadata value to your **event Region**:

```
{
"mcpServers": {
"aws-mcp": {
"command": "uvx",
"timeout": 100000,
"transport": "stdio",
"args": [
"mcp-proxy-for-aws==1.6.3",
"https://aws-mcp.us-east-1.api.aws/mcp",
"--metadata", "AWS_REGION=us-west-2"
]
}
}
}
```

Note

The `us-east-1` in the endpoint URL is the location of the MCP service itself and is correct as written. The `--metadata AWS_REGION` value is the Region the agent operates in — change that one to match your event.

The same configuration works on Windows as long as `uv` is installed and on your `PATH`. If Kiro reports that it cannot find

`uvx`, some MCP servers on Windows need the `uv tool run` invocation form instead — see the Windows configuration notes in the open source MCP servers for AWS repository.

1. Optionally install the AWS agent skills:

```
$ npx skills add aws/agent-toolkit-for-aws/skills
```

1. Restart Kiro or reconnect the server from the MCP Servers view.

Tip
If your team needs documentation access but not account access — for example on a locked-down machine with no AWS credentials — the AWS Knowledge MCP Server requires no authentication at all:

```
{
"mcpServers": {
"aws-knowledge-mcp": {
"url": "https://knowledge-mcp.global.api.aws"
}
}
}
```

Verify the connection

Start a new session in Kiro and ask:

```
What AWS Regions are available?
```

If you get a list of Regions, the server is connected and authenticated. Then confirm it can see *your* account:

```
Which AWS account am I connected to, and what Region are you operating in?
List any S3 buckets and Lambda functions that already exist.
```

If you get an authentication error, your AWS credentials have expired. Refresh them per Part 4 and reconnect the server.

Working with AWS through the agent

This is where the setup starts paying off. Three patterns cover most of what you will do over the next three days.

Ask for architecture guidance before you build. The agent has access to current AWS documentation and Well-Architected guidance, so it is a genuinely useful design reviewer:

```
I need to process uploaded PDFs, extract specific fields with Amazon Bedrock,
and let a reviewer work through a queue of results. Propose a serverless
architecture. Explain the tradeoffs and give me a rough monthly cost estimate
at 500 documents per month.
```

Have it verify against your real account rather than guessing. This is the single biggest advantage over an agent without AWS access:

```
Check whether the Bedrock models you referenced are actually available in my
Region and enabled for my account. If any are not, tell me which ones are.
```

Use it to troubleshoot with real data. Instead of pasting error messages back and forth, let it read the logs:

```
My Lambda function returned a 502. Find the function, read its recent
CloudWatch logs, identify the root cause, and explain what is wrong before
you change anything.
```

That last pattern is worth internalizing. Troubleshooting is where an agent with live account access is dramatically faster than one without, because it can read the actual failure instead of your summary of it. Deployment failures, permission errors, and timeouts are all much quicker to resolve this way.

Safety when the agent has AWS access

The agent now holds real credentials in a real account. Four rules:

1. **Read the plan before approving it.** Ask for a plan and a cost estimate before anything is created. Agents are decisive, and a decisive agent with these permissions can create a lot of resources quickly.
2. **Read commands before approving them.** Kiro asks before running commands and before destructive operations. That prompt is not a formality.
3. **Tag everything with your team name.** Cleanup on day three takes two minutes instead of twenty.
4. **Never point this at a production account.** A temporary hackathon account is the only appropriate place for this setup. When you take these techniques back to work, use a sandbox account and scope the credentials.

Tip

Write these constraints into a Kiro steering file at `.kiro/steering/aws-rules.md` and the agent applies them on every prompt without you restating them. Steering is covered in the **Building with Kiro** section of the workshop content.

Part 6: Deploy with infrastructure as code

Topics

Why infrastructure as code, even for a prototype

Clicking through the Console is faster for the first resource and slower for everything after. Define your infrastructure in code and you get four things that matter well before day three:

- **Repeatability.** Your teammate can deploy the same stack. So can your colleague next week, in your real account, after the workshop environment is gone.

- **Reviewability.** `cdk diff` shows exactly what a change will do before it happens.
- **Deletability.** One command removes everything. Manually created resources get orphaned and forgotten, which is how a workshop account ends up running a database nobody remembers creating.
- **Explainability.** Your template *is* the architecture document somebody will ask you for.

There is also a practical reason specific to this event: an agent is far more effective against declarative infrastructure than against Console clicks. Kiro can read your whole stack, reason about it, change it, and show you the diff.

Choosing a tool

Use the AWS CDK or AWS CloudFormation. Both are AWS-native, need nothing installed beyond what you already have, and are what the AWS Solutions Architects in the room can help you debug fastest.

	AWS CDK *(recommended)*	**AWS CloudFormation**	**Terraform**
You write	TypeScript, Python, Java, C#, Go	YAML or JSON	HashiCorp Configuration Language
Best for	Application infrastructure with real logic	Small, explicit stacks	Teams already standardized on it
Strengths	Sensible secure defaults, loops and conditionals, much less code	No extra tooling, full transparency, easy to read	Multi-cloud, large existing ecosystem
Watch out for	You should understand the CloudFormation it generates	Verbose; grows unwieldy fast	Separate state file to manage
State	Managed by CloudFormation	Managed by CloudFormation	You manage it

Recommendation. Choose the **CDK** if anyone on your team is comfortable in TypeScript or Python. A dozen lines of CDK produces a correct hundred-line CloudFormation template, including IAM roles and permissions you would otherwise have to write by hand — which is exactly the kind of work you do not want to spend a hackathon on.

Choose **CloudFormation** if your stack is small and explicit, or if you want to read every line of what gets created.

Terraform is fully supported on AWS and a legitimate choice, particularly if your agency has already standardized on it and you want work that transfers home directly. Two caveats for a hackathon: you manage state yourself, and you will find fewer people in the room able to help you debug it quickly. If your team knows Terraform well, use it. If you are learning it this week, do not.

Note

Whichever you choose, use **one** tool. No team has ever benefited from splitting a hackathon prototype across CDK and Terraform.

Quick start: AWS CDK

Prerequisites

- Node.js LTS installed. The CDK CLI requires Node even if you write your infrastructure in Python.
- Working AWS credentials from Part 4.

To install the CDK CLI

```
$ npm install -g aws-cdk
$ cdk --version
```

If you get a permission error on macOS or Linux and have administrator access, use `sudo npm install -g aws-cdk`.

To create and deploy your first CDK app

1. Create a project directory and initialize the app. The CDK CLI derives names from the directory name, so pick something meaningful and avoid spaces:

```
$ mkdir permit-triage && cd permit-triage
$ cdk init app --language typescript
```

For Python, use `--language python` and then activate the virtual environment:

```
$ cdk init app --language python
$ source .venv/bin/activate
$ python -m pip install -r requirements.txt
```

On Windows, activate with `.\.venv\Scripts\activate` instead.

1. Bootstrap your account and Region. This is a **one-time** setup per account and Region that creates the S3 bucket and roles the CDK uses for deployments:

```
$ cdk bootstrap
```

Run this from your project root and the CDK reads the account and Region from your environment automatically.

1. Confirm the CDK sees your stack:

```
$ cdk list
```

1. Preview the CloudFormation template your code produces:

```
$ cdk synth
```

Run this before every deployment. It catches synthesis errors in seconds and shows you exactly what AWS will be asked to create.

1. Deploy:

```
$ cdk deploy
```

The CDK prints a table of IAM changes and asks you to confirm. **Read it.** This is your last checkpoint before permissions are created in a real account.

1. After the first deployment, preview subsequent changes before applying them:

```
$ cdk diff
$ cdk deploy
```

1. When you are finished, remove everything:

```
$ cdk destroy
```

Tip
If you configured a named profile in Part 4 and did not set `AWS_PROFILE`, pass it explicitly: `cdk deploy --profile {{hackathon}}`.

Note
The CDK CLI uses the same credentials as the AWS CLI, and the `default` profile unless told otherwise. If `aws sts get-caller-identity` works, the CDK will work.

A useful set of commands to remember

Command	What it does
`cdk bootstrap`	One-time account and Region preparation
`cdk list`	Lists the stacks in your app
`cdk synth`	Generates the CloudFormation template; catches errors early
`cdk diff`	Shows what would change if you deployed now
`cdk deploy`	Creates or updates your resources
`cdk destroy`	Deletes the stack and its resources

Quick start: AWS CloudFormation

CloudFormation needs no additional tooling — the AWS CLI is enough.

To deploy a CloudFormation template

1. Write a template. This one creates a private, encrypted S3 bucket:

```
AWSTemplateFormatVersion: '2010-09-09'
Description: Document intake bucket for the hackathon prototype
Resources:
DocumentBucket:
Type: AWS::S3::Bucket
Properties:
BucketEncryption:
ServerSideEncryptionConfiguration:
- ServerSideEncryptionByDefault:
SSEAlgorithm: AES256
PublicAccessBlockConfiguration:
BlockPublicAcls: true
BlockPublicPolicy: true
IgnorePublicAcls: true
RestrictPublicBuckets: true
Tags:
- Key: team
Value: "{{your-team-name}}"
Outputs:
BucketName:
Description: Name of the document intake bucket
Value: !Ref DocumentBucket
```

1. Validate the syntax before deploying:

```
$ aws cloudformation validate-template --template-body file://template.yaml
```

1. Deploy it:

```
$ aws cloudformation deploy \
    --template-file template.yaml \
    --stack-name {{permit-triage}} \
    --capabilities CAPABILITY_IAM
```

`--capabilities CAPABILITY_IAM` is required whenever your template creates IAM roles or policies. Most useful stacks do.

1. Read the outputs:

```
$ aws cloudformation describe-stacks \
    --stack-name {{permit-triage}} \
    --query "Stacks[0].Outputs"
```

1. Delete it when you are done:

```
$ aws cloudformation delete-stack --stack-name {{permit-triage}}
```

Tip

When a stack fails, the reason is in the events, ordered newest first. The first `CREATE_FAILED` from the bottom is usually the real cause:

```
$ aws cloudformation describe-stack-events \
    --stack-name {{permit-triage}} \
    --query "StackEvents[?ResourceStatus=='CREATE_FAILED']"
```

Or simply ask Kiro to do it — with the AWS MCP Server connected, it reads the events itself and explains the failure.

Using Terraform

If your team already works in Terraform, it is a perfectly good choice. Notes specific to this event:

- Keep state local. A prototype in a temporary account does not need a remote backend. But commit your state file to your own repository along with the configuration, or the account will be deleted with the only record of what you deployed.
- Set the Region explicitly in your provider block so you cannot drift out of the event Region:

```
provider "aws" {
  region = "us-west-2"
}
```

- Terraform reads the same AWS credentials you configured in Part 4. If you used a named profile, add `profile = "{{hackathon}}"` to the provider block or set `AWS_PROFILE`.
- The usual loop: `terraform init`, `terraform plan`, `terraform apply`, and `terraform destroy` at cleanup.
- Run `terraform destroy` before the 72-hour window closes. Terraform-created resources are not tracked by CloudFormation, so they will not show up when a facilitator reviews stacks at teardown.

Let Kiro write your infrastructure

You do not have to write any of this from scratch. With the AWS MCP Server connected, Kiro authors, deploys, and debugs infrastructure code directly — and because it has live documentation access, it writes against current APIs rather than whatever was in its training data.

To have Kiro build your stack

```
Create a CDK app in TypeScript for this project.

Requirements:
- An S3 bucket for uploaded documents. Private, encrypted, versioned.
- A Lambda function triggered on upload that calls Amazon Bedrock to extract
  required fields, then writes results to DynamoDB.
- An HTTP API through API Gateway for a reviewer to list and read results.

Constraints:
- Region us-west-2.
- Least-privilege IAM. No wildcard resources in policies you write.
- Tag every resource with team={{your-team-name}}.
- Simplest thing that works. This is a hackathon prototype.
```

```
Show me the CDK code and the output of `cdk synth` before deploying anything.
```

Then, before you approve the deployment:

```
Walk me through what this creates, what it will cost per month at low volume,
and anything in here that would be unacceptable in production.
```

That second prompt is worth the thirty seconds. It gives you an accurate description of your own architecture and an honest list of its gaps — both of which you need at demo time.

Practices worth keeping

These carry over from the hackathon to real work.

Deploy early and often. Get an empty stack deployed in the first hour of day one. Discovering a credentials or bootstrap problem on day one is a minor annoyance; discovering it an hour before demos ends your demo.

Tag everything. `team`, `project`, and `event` at minimum. Cleanup and cost attribution both depend on it.

Secure defaults from the first commit. Encryption on, public access blocked, least-privilege IAM. It is no slower than doing it wrong, and in this sector somebody will ask.

Never commit secrets. No credentials, no API keys, no connection strings in your repository. Use environment variables during the event, and AWS Secrets Manager or Parameter Store for anything real. This matters more than usual here, because you are pushing this repository to GitHub before the account is deleted.

Let CloudFormation own its resources. Do not hand-edit a CDK- or CloudFormation-managed resource in the Console. It causes drift, and the next deployment will either revert your change or fail confusingly.

Start simple and add complexity only when you need it. This is AWS's own top CDK guidance and it applies to your whole build. You can refactor as requirements emerge; you do not need to design for every scenario up front. Three days is enough time to over-engineer something, so watch for it.

Part 7: Troubleshooting reference

Symptom	Likely cause and fix
Kiro sign-in rejects `KiroProPassword`	The one-time password was already used. Reset it in IAM Identity Center in `us-east-1`. See Troubleshooting sign-in.
Kiro signs in but shows no Pro subscription	License assignment incomplete. Ask a facilitator to check the `kiro` user's subscription.
`Unable to locate credentials`	Re-paste your short-term credentials, or run `aws login`. See Renewing access during the event.
`aws configure sso` finds no accounts	Expected. The event Identity Center instance issues your Kiro license only. Get AWS credentials from the

	Workshop Studio panel instead.
`ExpiredToken`	Session expired. Sign in again.
Kiro's AWS tools return authentication errors	Your AWS credentials expired. Refresh, then reconnect the MCP server.
MCP server will not connect	Verify `uv --version` works. Reconnect from the MCP Servers view, then restart Kiro.
`AccessDenied` on a reasonable action	Check the Region first. Then check the guardrails list.
Resources you created are not visible	Region mismatch. Check the Console Region selector and `aws configure get region`.
`cdk bootstrap` fails	Confirm credentials with `aws sts get-caller-identity` and confirm you are in the event Region.
`--app is required either in command-line, in cdk.json...`	You are not in your CDK project directory. `cd` into it.
`cdk deploy` fails on IAM	Add `--capabilities CAPABILITY_IAM` for raw CloudFormation. For the CDK, read the IAM change table and confirm.
Bedrock returns `AccessDeniedException`	Model access may not be enabled for that model in your Region. Ask Kiro which models are actually available to your account.
Bedrock returns `ThrottlingException`	You are sending requests too fast. Keep to roughly 1 request per second and add retries with backoff.
Lambda returns 502 or times out	Read the CloudWatch logs. Ask Kiro to find the function and diagnose it.

When you are stuck for more than thirty minutes, ask a Solutions Architect. They are in the room specifically for this, and they have seen your problem before. Thirty minutes is the threshold; do not spend half a day proving you can solve it alone.

Part 8: Save your work and clean up

Warning

Your workshop AWS account is deleted when the 72-hour window closes. The account, every resource in it, and anything stored only on the provisioned browser environments goes with it. There is no extension, no snapshot, and no recovery.

Read this section on day one.

Topics

- Save your work
- What to save
- Clean up your AWS resources

Save your work

Do this continuously, not at the end. The most common regret from these events is a team with a working prototype and nowhere to put it.

Important
If you are working in one of the provisioned browser environments — hosted VS Code or the hosted Kiro desktop — your files live on an EC2 instance inside the workshop account. That instance is deleted with the account. Committing locally on that machine is **not** saving your work. It has to leave the environment.

To push your work to your own GitHub repository

1. On day one, create a repository under your own GitHub account or your organization's. Decide public or private now — if public, be certain no credentials or synthetic-but-realistic-looking data ever gets committed.
2. Set your Git identity in the environment you are working in:

```
$ git config --global user.name "{{Your Name}}"
$ git config --global user.email "{{you@example.gov}}"
```

1. Initialize the repository and add a `.gitignore` before your first commit. Ask Kiro to generate one appropriate to your stack, then confirm it excludes environment files, credentials, build output, and `cdk.out`:

```
$ git init
$ git add .gitignore
$ git commit -m "Add gitignore"
```

1. Add your remote and push. Use a GitHub personal access token or SSH key — GitHub does not accept account passwords over HTTPS:

```
$ git remote add origin https://github.com/{{your-org}}/{{your-repo}}.git
$ git branch -M main
$ git push -u origin main
```

1. **Push again at the end of every working session.** Not just at the end of day three.

```
$ git add -A && git commit -m "{{what changed}}" && git push
```

To save to your local machine instead

If GitHub is not an option — some organizations restrict it — get the files off the workshop environment another way:

- Working locally already? Your files are already safe. Confirm they are outside any workshop-provisioned directory.

- Using hosted VS Code? Right-click the project folder in the Explorer and choose **Download**. Do this for the whole project, and verify the archive actually opens.
- Either way, keep a copy somewhere backed up. A single download in your `Downloads` folder is not a backup.

Warning
Scan your history before pushing anywhere public. Short-term AWS credentials, API keys, and connection strings must not be in your repository. Ask Kiro to check:

```
Scan this repository, including files staged for commit, for anything that
looks like a credential, access key, session token, connection string, or
API key. Also flag any data file that might contain real records rather
than synthetic ones. Report findings; do not change anything yet.
```

What to save

Save more than the application. Two of these three outlive the prototype.

Your source code. The obvious one.

Your `.kiro/` directory. This is the highest-value artifact you produce, and the easiest to forget. Your steering files, specs, hooks, and agent configuration are what let you work this way again on Monday against your real codebase. The prototype is a demo; the working method is what changes how your team operates. Confirm `.kiro/` is committed and not caught by your `.gitignore`.

Your infrastructure as code. Your CDK app, CloudFormation templates, or Terraform configuration *is* your architecture, in a form somebody can actually redeploy. This is the difference between "we built a thing at a hackathon" and "here is a running proof of concept you can stand up in your own account."

Also worth capturing, in a `NOTES.md` in the repository

- The three hardest problems you hit and how you solved them.
- Your honest list of gaps — what would need to change before this touched real data. If you used the `// PROTOTYPE:` convention, `grep` for it and paste the results in.
- Three moments Kiro surprised you, good and bad. That is your real assessment of where agentic development helps today, and it is worth more to your organization than the prototype.
- Rough monthly cost at realistic volume. This is the number people ask about first.

Clean up your AWS resources

The environment shuts down on its own, so formal cleanup is optional. Do it anyway. It is good practice, it is how you are expected to behave in a real account, and it forces you to notice anything you created and forgot.

Note
Do this only after your work is pushed and verified. Deleting stacks does not delete your source code, but there is no reason to take the risk in the wrong order.

To clean up

1. Delete your infrastructure with the tool that created it:

```
$ cdk destroy
$ aws cloudformation delete-stack --stack-name {{permit-triage}}
$ terraform destroy
```

1. Check for anything left behind. If you tagged your resources, this is quick:

```
$ aws resourcegroupstaggingapi get-resources \
    --tag-filters Key=team,Values={{your-team-name}}
```

1. Empty and delete any S3 buckets that blocked stack deletion. CloudFormation cannot delete a bucket that still has objects in it:

```
$ aws s3 rm s3://{{bucket-name}} --recursive
$ aws s3 rb s3://{{bucket-name}}
```

1. Confirm nothing is still running that you did not expect:

```
$ aws cloudformation list-stacks \
    --stack-status-filter CREATE_COMPLETE UPDATE_COMPLETE
```

Tip
Ask Kiro to do the audit for you while you finish writing up. With the AWS MCP server connected it can enumerate resources across services and tell you what is left:

```
List everything currently deployed in this account, grouped by service.
Flag anything that would keep costing money if this account were not
being deleted. Do not delete anything yet.
```

Related resources

Kiro

- Kiro documentation
- Kiro downloads
- Kiro CLI documentation
- Authentication
- MCP configuration

AWS credentials and CLI

- Installing or updating to the latest version of the AWS CLI
- Authenticating with short-term credentials
- Sign in through the AWS CLI

Connecting agents to AWS

- Agent Toolkit for AWS
- Getting started with the Agent Toolkit
- Setting up the AWS MCP Server

- Open source MCP servers for AWS

Infrastructure as code

- Getting started with the AWS CDK
- Tutorial: Create your first AWS CDK app
- Best practices for developing cloud infrastructure with the AWS CDK
- AWS CloudFormation User Guide
- Construct Hub
- Terraform AWS Provider

Services

- Amazon Bedrock
- AWS Lambda
- Amazon S3
- Amazon DynamoDB
- Amazon API Gateway
- Amazon Textract

Practices

- AWS Well-Architected Framework
- IAM security best practices
- AI-Driven Development Life Cycle

Open data for synthetic and public data sets

- data.gov
- IPEDS and NCES
- Census data
- Registry of Open Daata on AWS